ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS PROYECTO INICIAL 2026-2 [:)]

Ciclo No. 1-2-3-4-5/4 — *Refactoring* y Extensión

El proyecto inicial tiene como propósito desarrollar una aplicación que permita simular una situación inspirada en el **Problem I** de la maratón de programación internacional **2025 Slot Machine**. En este simulador los símbolos deben ser de colores diferentes.

CUARTO CICLO

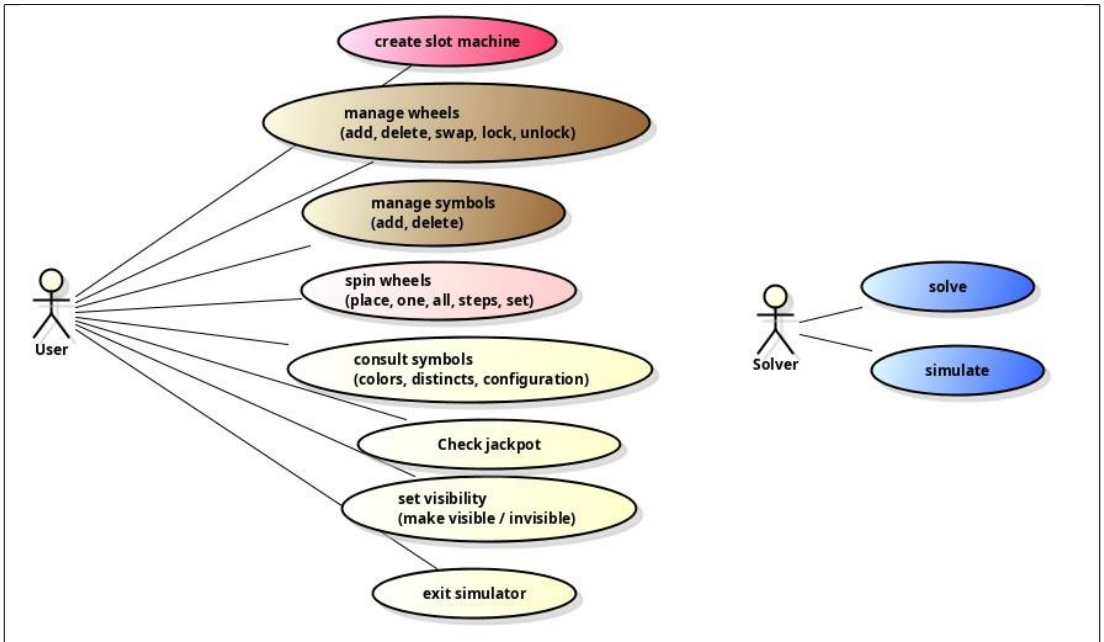
Los requisitos para este ciclo de desarrollo están indicados a continuación. Siempre hay un requisito implícito: el de **EXTENSIBILIDAD**.

MUY IMPORTANTE PLANIFICAR LOS MINI-CICLOS PARA ORIENTAR EL DESARROLLO.

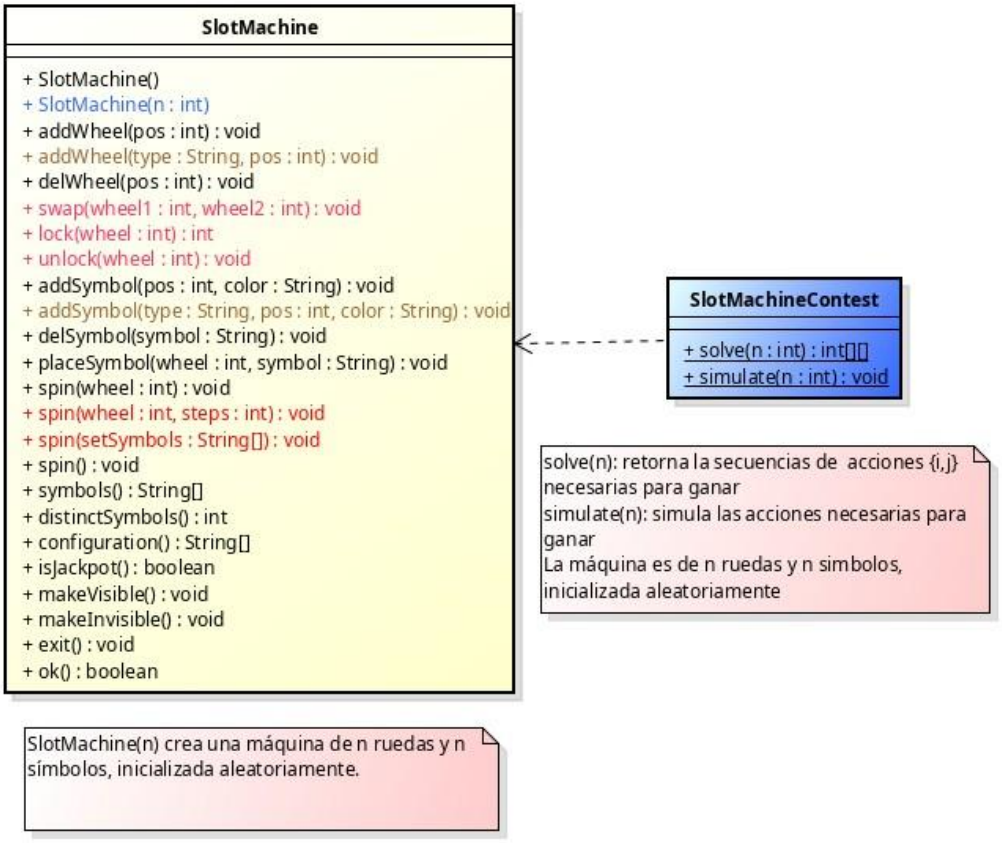
REQUISITOS FUNCIONALES

El simulador debe permitir:

16. Manejar diferentes tipos de ruedas y de símbolos.
17. Ofrecer mínimo tres tipos de ruedas: **normal** (la que tenemos), **lefty**(si hay una rueda a su izquierda, al girar copia su estado), **rebel**(no se deja bloquear, ni intercambiar, ni eliminar).
18. Ofrecer mínimo tres tipos de símbolos: **normal** (el que tenemos), **ephemeral** (en cada giro va decrementando su tamaño hasta quedar como un punto), **shy** (alterna su estado de visible a invisible, cada vez que es seleccionado en la rueda).
19. Ofrecer un nuevo tipo de alguno de los elementos. (Propuesto por ustedes).



REQUISITOS DE DISEÑO



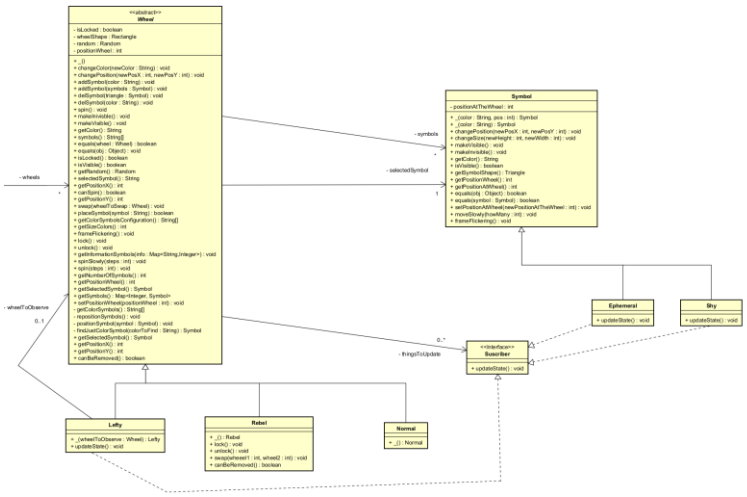
REQUISITOS DE USABILIDAD

1. Los elementos de diferentes tipos deben poder distinguirse claramente visualmente.

REQUISITOS DE ENTREGA

Los productos de esta entrega son:

1. Diseño completo en la herramienta astah



2. Implementación siguiendo los estándares de codificación y documentación de java.
3. Casos de pruebas de unidad de su proyecto: SlotMachineC4Test
4. Mínimo dos casos de prueba de unidad compartidos en la clase SlotMachineCC4Test
5. Dos pruebas de aceptación preparadas para la presentación.
6. Documento de retrospectiva del proyecto. (Ver ciclo uno)
   Es necesario incluir la retrospectiva de este ciclo y de los anteriores.
   DEFINIR EL ESTADO EN TERMINOS DE LOS MINI-CILOS PLANIFICADOS

#Pregunta	Pregunta	Ciclo1	Ciclo 2	Ciclo 3	Ciclo 4
	¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.
	¿Cuáles el estado actual del proyecto en términos de mini-ciclos? ¿por qué?	Completo.	Completo.	Completo.	Completo
	¿Cuál fue el tiempo total invertido por cada uno de	15 horas cada uno.	18 horas cada uno.	18 horas cada uno.	18 horas cada uno.

	ustedes? (Horas/Hombre)				
	¿Cuál consideran fue el mayor logro? ¿Por qué?	Aplicar la práctica XP llamada Pair Programming.	Aplicar la práctica TDD porque está al aplicarse brinda cierta seguridad en que las funcionalidades que se exigen en el proyecto se cumplan.	El mayor logro fue establecer pruebas para depurar el programa.	El mayor logor fue establecer un correcto usuo de la herencia sin necesidad de violar el principio de responsabilidad única de cada clase
	¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?	Utilizar GitHub, esto se fue aprendiendo a medida que se utilizaba para guardar el proyecto.	Entender los requerimientos que se pedían, lo que se hizo fue hablar con la profesora para asegurar lo que se creía de cada requerimiento.	El mayor logro fue hacer las pruebas para el juez interactivo, se hicieron las pruebas a mano con toda la estructura esperada y lo pase a txt.	El mayor logro fue respetar e implementar la herencia respetando la estructura da en el laboratorio. Lo solucionamos creando objetos por nombre de clase.
	¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?	Comunicarnos.	Lo que pudimos hacer fue redistribuirnos trabajo y explicarnos de manera sencilla lo que hizo cada uno.	N-N	N-N

	Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?	Simplicidad, Facilito el proceso de programar solo lo esencial y después poderlo extender.	La comunicación con el cliente o profesor fue muy necesaria para cumplir los requisitos y esta es una de las prácticas XP, estar en constante diálogo con el cliente.	Simplicidad, Facilito el proceso de programar solo lo esencial y después poderlo extender.	Simplicidad, Facilito el proceso de programar solo lo esencial y después poderlo extender.
	¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.	**Referencias:** Google. (2026). *Gemini* (Gemini Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño]. https://gemini.google.com Anthropic. (2026). *Claude* (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Model	**Referencias:** Martin, R. C. (2012). *Código limpio: Manual de estilo para el desarrollo ágil de software* (2.ª ed.). Anaya Multimedia. Google. (2026). *Gemini* (Gemini Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño].	**Referencias:** • Escuela Colombiana de Ingeniería Julio Garavito. (2026). *Requisitos de entrega - Proyecto DOPO*. • https://open.kattis.com/problems/slotmachine?editreqsubmit=20521086	**Referencias:** • Apache Software Foundation. (s.f.). *Apache Maven Project* (Versión 3.10.0) [Software]. Recuperado el 21 de octubre de 2026, de https://maven.apache.org/ • EclEmma. (2023). *JaCoCo Java Code Coverage Library* (Versión 0.8.10) [Software]. Recuperado de https://www.jacoco.org/jacoco/ • Escuela Colombiana

		o de lenguaje de gran tamaño]. https://claude.ai	https://gemini.google.com Anthropic. (2026). *Claude* (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Modelo de lenguaje de gran tamaño]. https://claude.ai		de Ingeniería Julio Garavito. (2026). *Requisitos de entrega - Proyecto DOPO* . • Microsoft. (s.f.). *Visual Studio Code* [Software]. Recuperado el 21 de octubre de 2026, de https://code.visualstudio.com/ • Oracle Corporation. (s.f.). *Java Platform, Standard Edition 8* (Versión 1.8.0_504) [Software]. Recuperado el 21 de octubre de 2026, de https://docs.oracle.com/javase/8/docs/ • PMD. (s.f.). *PMD Source Code Analyzer* (Versión 3.21.2) [Software]. Recuperado el 21 de octubre de 2026, de https://pmd.github.io/

Desarrollo del ciclo 1:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☑ ~~SlotMachine()~~ ☑ ~~addWheel(int pos)~~ ☑ ~~delWheel(int pos)~~ ☑ ~~addSymbol(int pos, String color)~~ ☑ ~~delSymbol(String symbol)~~	☑ ~~Wheel()~~ ☑ ~~changColor(color: String)~~ ☑ ~~changePositionX(pos: int)~~ ☑ ~~changePositionY(pos: int)~~	☑ ~~Symbol(color: string)~~ ☑ ~~changePositionx(pos: int)~~ ☑ ~~changePositiony(pos: int)~~
2	☑ ~~symbols()~~ ☑ ~~placeSymbol(int wheel, String symbol)~~ ☑ ~~distinctSymbols()~~ ☑ ~~configuration()~~	☑ ~~addSymbol(Symbol: String)~~ ☑ ~~delSymbol(Symbol: String)~~ ☑ ~~selectSymbol()~~	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~
3	☑ ~~spin(int wheel)~~ ☑ ~~spin()~~ ☑ ~~isJackpot()~~ ☑ ~~configuration()~~ ☑ ~~ok()~~ ☑ ~~exit()~~	☑ ~~spin()~~ ☑ ~~symbols()~~	
4	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~	☑ ~~makeVisible()~~ ☑ ~~makeInvisible()~~	

Desarrollo del ciclo 2:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☑ ~~swap(wheel1 : int, wheel2 : int)~~ ☑ ~~lock(wheel : int)~~ ☑ ~~unlock(wheel : int)~~	☑ ~~agregar atributo isLocked y~~ ☑ ~~agregar lógica de spin~~	
2	☑ ~~spin(wheel : int, steps : int)~~ ☑ ~~spin(setSymbols : String[])~~	☑ ~~Spin(steps:int)~~ ☑ ~~spinSlowly(steps:int)~~	

Desarrollo del ciclo 3:

#Miniciclo	SlotMachineContest	Pruebas	
1	~~printOI(int i, int u, SlotMachine machine)~~ ~~different(SlotMachine machine)~~	~~Establecer prueba de depuracion~~	
2	~~identical(SlotMachine machine)~~ ~~simulate(int n)~~ ~~solve(int n)~~		

Desarrollo del ciclo 4:

#Miniciclo	SlotMachine	Wheel e hijas	Symbol e hijas
1		⊟ ~~public Lefty()~~ ⊟ ~~Public Rebel()~~	⊟ ~~public Shy(String color)~~ ⊟ ~~Ephemeral(String color)~~ ⊟ ~~Rotation(String color)~~

2		~~canSpin()~~ ~~spin(Wheel leftWheel)~~ ~~lock()~~ ~~unlock()~~ ~~swap(Wheel wheelToSwap)~~	~~efect()~~ ~~reduce()~~ ~~rotating()~~
3	~~addWheel(String type, int pos)~~ ~~spin(int wheel)~~		
4	~~spin(int wheel, int steps)~~ ~~spin(String[] setSymbols)~~		

Los productos los deben publicar en un repositorio Git y deben publicar en Moodle un archivo txt que contenga la dirección URL. El nombre del archivo .txt debe ser la concatenación de los primeros apellidos más la primera letra del segundo apellido de los autores, ordenados alfabéticamente. Publicar productos a revisión: Semana 10 - 3 octubre

ESCUELA COLOMBIANA DE INGENIERÍA

DESARROLLO ORIENTADO POR OBJETOS

PROYECTO INICIAL 2026-2

Ciclo No. 5/4 — *Refactoring* y Extensión

El proyecto inicial tiene como propósito desarrollar una aplicación que permita simular una situación inspirada en el **Problem I** de la maratón de programación internacional **2025 Slot Machine**. En este simulador los símbolos deben ser de colores diferentes.

CIERRE

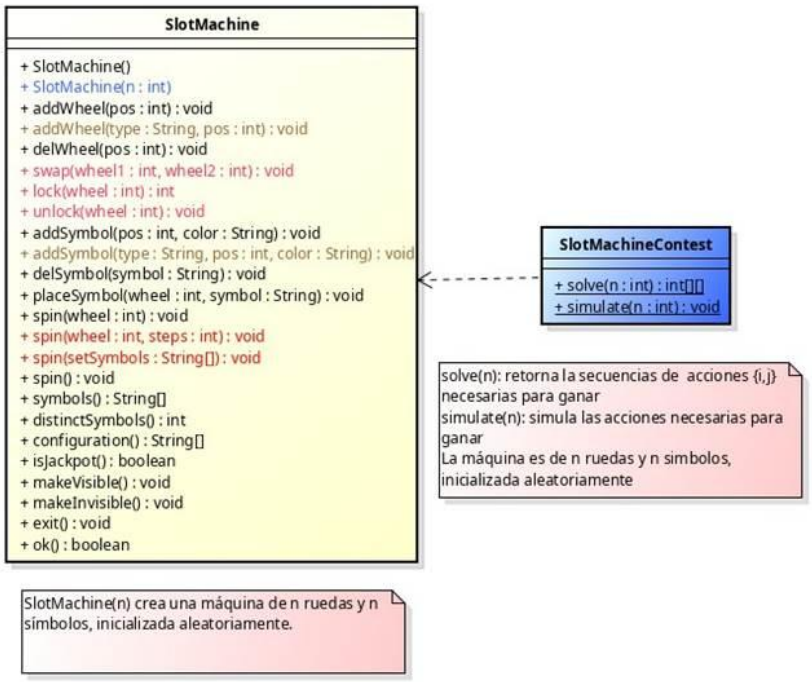
El objetivo de esta entrega es evaluar y perfeccionar la calidad del simulador mediante el uso de herramientas modernas de desarrollo.

REQUISITOS FUNCIONALES

Para eso se espera que en esta entrega se tenga:

1. El proyecto en IDE VS Code
2. Informe de análisis dinámico: informe de cubrimiento de pruebas de unidad generado con JaCoCo.
3. Informe de análisis estático: informe de cumplimiento de reglas de calidad para java generado con PMD.

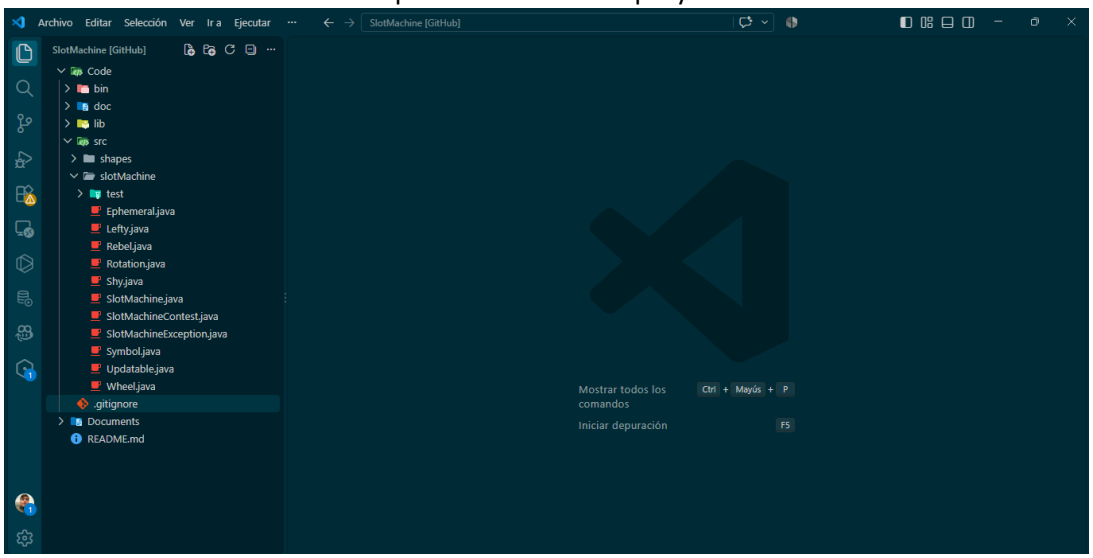
REQUISITOS DE DISEÑO



ENTREGA

Los productos de esta entrega son:

1. El proyecto en IDE VS Code
   - Pasando a Visual Studio Code se puede evidenciar el proyecto



2. Informe de análisis dinámico: informe de cubrimiento de pruebas de unidad generado con JaCoCo. Después de instalar Apache Maven y agregar una estructura a aquel con un archivo llamado pom.xml para generar el informe se puede ver el informe en el archivo dentro del repositorio en GitHub en C:\Users\samut\git\SlotMachine\Code\target\site\jacoco\index.html se puede ver el informe igual que en lo siguiente

slotmachine

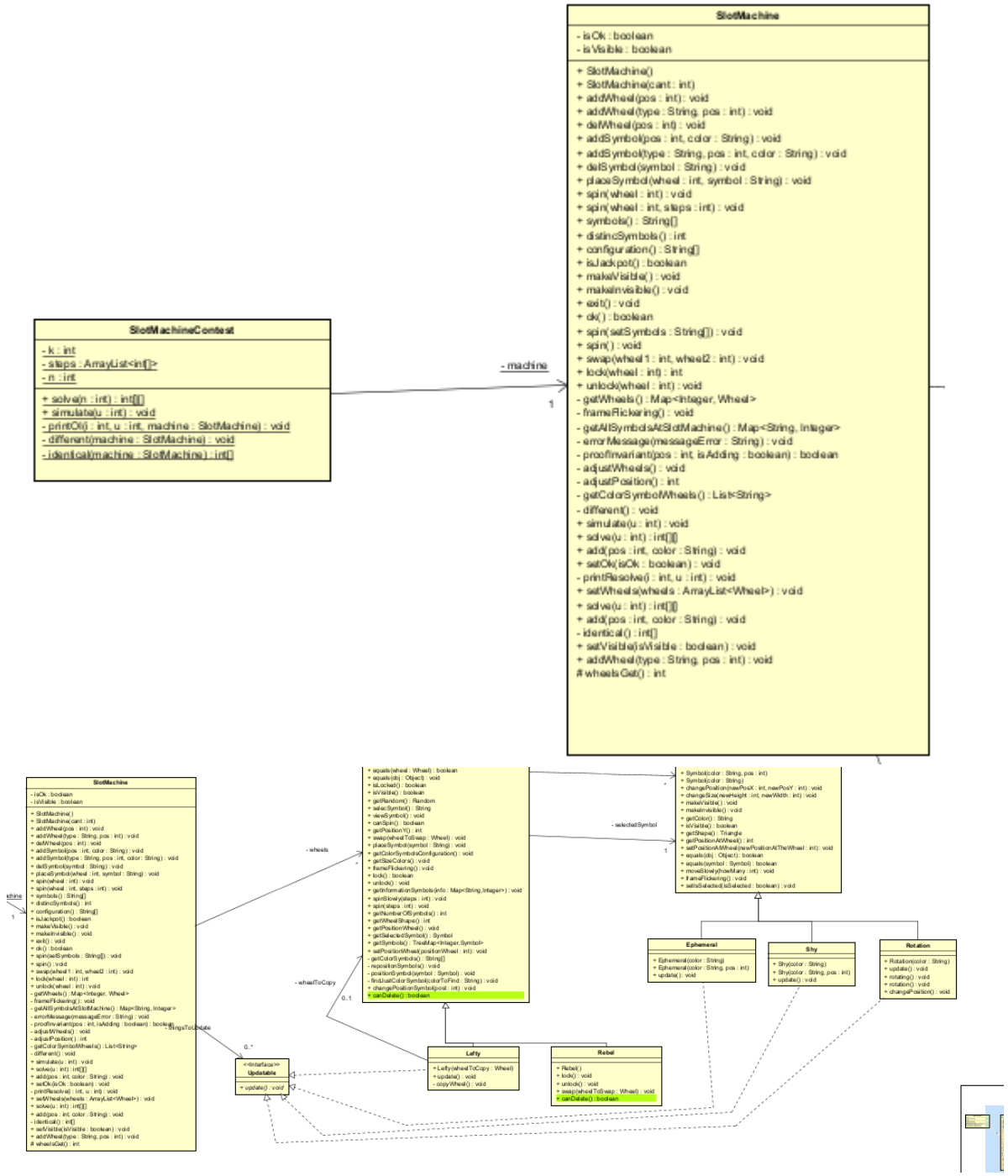
slotmachine

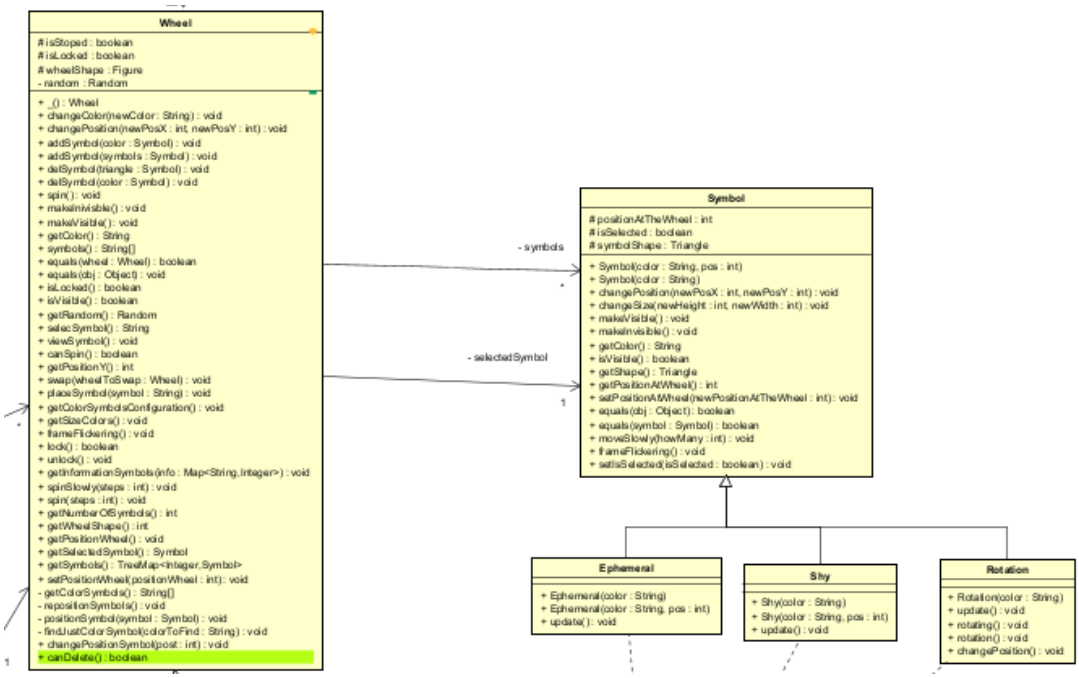
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
slotMachine		77%		59%	147	325	190	792	31	128	3	10
shapes		89%		70%	20	64	49	278	15	52	0	7
Total	887 of 4.521	80%	167 of 418	60%	167	389	239	1.070	46	180	3	17

En este análisis se puede decir que más del 75% de la lógica de dominio la cual es de la carpeta slotMachine está cubierta o probada dinámicamente.

3. Informe de análisis estático: informe de cumplimiento de reglas de calidad para java generado con PMD.
   Este archivo que se va a mostrar en capturas se puede encontrar .\SlotMachine\Code\target\reports\pmd.html, se resalta que se ordenan por prioridad y no se evidencia ninguna falta de prioridad 1, se puede evidenciar el análisis estático en el archivo antes mencionado.

4. Diseño completo en la herramienta astah





Se puede ver mejor evidenciado en Astah.

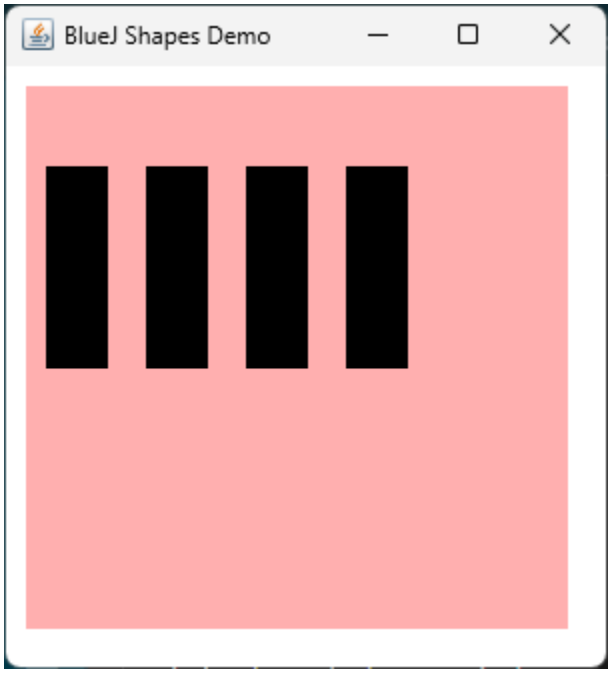
5. Implementación siguiendo los estándares de codificación y documentación de java en VS Code
6. Casos de pruebas de unidad de su proyecto en VS Code : SlotMachine*Test: 1,2,3,4,Contest
   Lo que aparece al ejecutar los test es lo siguiente

```
[INFO] Running slotMachine.test.Problem
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
10.05 s -- in slotMachine.test.Problem
[INFO] Running slotMachine.test.SlotMachineC1Test
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.170 s -- in slotMachine.test.SlotMachineC1Test
[INFO] Running slotMachine.test.SlotMachineC2Test
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.058 s -- in slotMachine.test.SlotMachineC2Test
[INFO] Running slotMachine.test.SlotMachineC4Test
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.055 s -- in slotMachine.test.SlotMachineC4Test
[INFO] Running slotMachine.test.WheelTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.032 s -- in slotMachine.test.WheelTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 46, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.10:report (report) @ slotmachine ---
[INFO] Loading execution data file
C:\Users\samut\git\SlotMachine\Code\target\jacoco.exec
[INFO] Analyzed bundle 'slotmachine' with 17 classes
```

```
[INFO] ------------------------------------------------------------------------
-------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
-------
[INFO] Total time:  03:22 min
[INFO] Finished at: 2026-10-22T17:15:26-05:00
[INFO] ------------------------------------------------------------------------
-------
```

Esto al correr en el directorio Code/src/ el siguiente comando con una terminal PowerShell
.\mvnw.cmd test

7. Una prueba de aceptación ejecutando en VS Code.
   Se ejecuta la prueba, de SlotMachineAcceptanceTest, **shouldntDeleteWheelWhenIsLocked**, se muestra su ejecución a continuación



Ahora se observa que corre esta prueba correctamente como se ve a continuación

```
[INFO] Running slotMachine.test.SlotMachineAceptanceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 107.0 s -- in slotMachin
test.SlotMachineAceptanceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

8. Informe de análisis dinámico; resultado inicial y resultado final.
   Descripción de las decisiones tomadas después de estudiar el resultado inicial. Meta: más de 75 % de cubrimiento de código.

   En anteriores capturas se observa que cubre el 77% de todo el código de dominio que en este caso es la carpeta de slotMachine y en la presentación, shapes, cubre un 89%, en promedio 80%.

9. Informe de análisis estático: resultado inicial y resultado final.
   Descripción de las decisiones tomadas después de estudiar el resultado inicial. Meta: cumplimiento de todas las reglas de prioridad alta (rojo).

   En este caso no se vieron errores de prioridad 1 que son la prioridad roja e incluso con la siguiente línea en el archivo pom.xml se verifica que el reporte solamente se haga de prioridad 1 y 2, no incluye prioridad 3, 4 y 5.

```
<minimumPriority>2</minimumPriority>
```

10. Documento de retrospectiva del proyecto. (Ver ciclo uno)
    Este ciclo 5 está completo en todo su alcance y las referencias que se usó para este ciclo se colocó en la sección del ciclo 4.

Los productos los deben publicar en Moodle un archivo zip que contenga todos los productos. El nombre del archivo .zip debe ser la concatenación de los primeros apellidos más la primera letra del segundo apellido de los autores, ordenados alfabéticamente.

Publicar productos a revisión: Semana 12 - 24 octubre